Aethon OS

Master Architecture & Build Roadmap

Aethon Studios

Architecture reference - 25 September 2026

Status: Living source-of-truth document
Purpose: Define the long-term architecture, product vision, locked operating rules, implementation roadmap, and current build-state tracking for Aethon OS.
Primary user today: Saad / Aethon Studios
Initial deployment: Single-user, local Windows desktop application
Future deployment: Multi-user internal agency platform with shared data and role-based access
Last updated: 25 September 2026

1. How Codex Must Use This Document

This file is the canonical product and architecture reference for Aethon OS.

Before any major implementation task, Codex must:

Read this document.

Inspect the current repository and running application.

Compare the current implementation against the architecture and roadmap below.

Identify what is:

Implemented

Partially implemented

Missing

Implemented incorrectly

No longer aligned with the architecture

Avoid rebuilding features that already exist correctly.

Avoid implementing later-stage features before the foundations they depend on are stable.

Preserve working behaviour unless a deliberate refactor is required.

Update the **Implementation Status** and **Build History** sections after meaningful milestones.

Never silently change a locked business rule in this document. If a rule must change, clearly flag the conflict and propose the change before implementing it.

Prefer incremental delivery over attempting to build the entire platform in one pass.

This document describes the **end goal**. It is not permission to implement everything immediately.

The development philosophy is:

Build a clean foundation, use it in real Aethon operations, observe friction, improve it, automate the proven workflow, and repeat.

2. Product Vision

Aethon OS is the internal operating system for **Aethon Studios**.

Aethon currently operates across many separate platforms, including:

Notion

Google Drive

Google Docs / Sheets

Gmail

Google Calendar

Google Meet

Granola

Frame.io

Canva

GoHighLevel

Xero

Meta Ads Manager

Meta Business Suite

Instagram

Facebook

TikTok

LinkedIn

YouTube / YouTube Studio

Time Doctor

CapCut

AI tools such as ChatGPT / Codex / Gemini

Aethon OS is **not intended to replace every specialist platform**.

Instead, it becomes the operating layer that connects them and reduces fragmentation.

The system should eventually let Aethon manage:

Client commitments

Retainers

One-off projects

Community work

Deliverables

Content production

Shoots

Editing

Reviews

Client approvals

Publishing

Client reporting

Tasks

Priorities

Meetings

Emails

CRM

Leads

Paid ads

Finances

Team workload

Time usage

Business health

Forecasting

Content intelligence

AI-assisted operations

The long-term goal is for Aethon leadership to spend as little time as possible on repetitive administration.

Human time should increasingly be spent on:

Strategy

Creative judgement

Sales

Client relationships

Major decisions

Exception handling

---

3. Core Product Principle

The most important object in Aethon OS is:

The promise made to the client.

The system should continuously answer:

What did we promise?

What service period does it belong to?

What has been produced?

What has been delivered?

What is still outstanding?

What is waiting on the client?

What is at risk?

What has carried over?

What have we over-delivered?

What should be worked on next?

Therefore:

Deliverables are the spine of Aethon OS.

Everything else - tasks, production, editor workload, reporting, client health, publishing, finance, and AI - should connect back to delivery where relevant.

---

4. Current Deployment Strategy

4.1 Initial phase

Aethon OS is initially:

Windows-first

Single-user

Local-first

Running on Saad's PC

Using a local database

Not publicly hosted

Not multi-user

Not cloud-dependent

Recommended stack:

Tauri 2

React

TypeScript

Vite

SQLite

Rust/Tauri backend

Local file access

Local background worker/processes where appropriate

4.2 Future phase

Later, Aethon OS may become multi-user.

Possible future users:

Saad

Nazif

Editors

Virtual assistants

Media buyers

Clients through a limited portal

Future multi-user architecture must not require rewriting the entire UI.

Therefore:

Keep persistence/database access separated from UI components.

Keep domain logic separated from storage details.

Avoid assumptions that only one user can ever exist, even though authentication is not required initially.

Avoid hard-wiring Windows-specific application behaviour into business logic where unnecessary.

4.3 Heavy processing

Aethon OS will eventually process:

Large video files

Audio

Transcripts

Local footage folders

AI media analysis

Heavy media operations should be suitable for a **local worker running on the main PC**.

Later, that worker could run on an always-on office machine if required.

---

5. Engagement Types

A client can have multiple engagements.

Supported engagement types:

5.1 Retainer

Recurring work, usually monthly.

Examples:

8 videos per month

Organic content management

Meta Ads management

Monthly reporting

5.2 Project

Finite one-off work.

Examples:

One-off campaign

One-off shoot

Standalone ad campaign

Event production

Projects may use milestones as well as deliverables.

Example:

Pre-production

Shoot

First cut

Revisions

Final delivery

Campaign launch

5.3 Community

Community / not-for-standard-commercial-work engagements.

Community work should use the same production infrastructure where useful:

Content

Shoots

Editing

Reviews

Deadlines

Tasks

But may exclude or simplify:

Profitability

Sales pipeline logic

Commercial overservicing analysis

Billing

---

6. Contract & Deliverable Intake

Long-term desired workflow:

Contract is uploaded.

AI reads the contract.

AI extracts:

Services

Quantities

Frequency

Service periods

Deliverable rules

Special conditions

Aethon displays proposed deliverables.

Human reviews and edits.

Human activates the engagement.

Aethon creates operational deliverable definitions and service-period obligations.

Important rule:

Contract extraction must never silently activate obligations without human review.

In the early version, deliverables may be created manually.

AI contract parsing is a later milestone.

---

7. Deliverable Measurement Types

Aethon must not treat every quantity the same.

7.1 Exact Quantity

Example:

8 organic videos per month

Behaviour:

Full quantity is required.

Remaining quantity is operationally meaningful.

Missing quantity can become overdue/carryover.

7.2 Flexible Pool

Example:

8 videos per month consisting of any combination of organic and paid-ad videos

Behaviour:

Total quantity matters.

Sub-type split is not contractually fixed.

Each content item still has its own usage classification.

Example for SZ Engineers:

8 videos total

Could be:

6 organic

2 paid

Or another mix

As long as total contractual pool is satisfied

7.3 Fixed Sub-Quotas

Example:

Ride Inspect:

8 organic videos

4 dedicated ads

These are tracked as separate contractual buckets.

A single piece of content should not automatically satisfy two contractual obligations.

7.4 As-Needed Ceiling

Example:

Up to 10 photos

Meaning:

10 is a ceiling, not a target.

Fewer than 10 does not mean Aethon is behind.

Display conceptually:

4 used / 10 maximum

Not:

4 / 10 incomplete

7.5 Ongoing Monitored Service

Examples:

Meta Ads management

Comment moderation

Social management

These services do not become "complete" like a video quota.

Instead, Aethon should monitor:

Activity

Performance

Anomalies

Failures

Required intervention

---

8. Deliverable Rules

8.1 Definition of Delivered

For content deliverables:

Client approval = Delivered

The content still continues through:

Scheduling

Publishing

But the contractual delivery obligation is considered satisfied once the client approves the creative.

8.2 Carryover

If a monthly obligation is not fully delivered:

Example:

September requirement: 8 videos
September delivered: 7

Then October must show:

October contractual requirement: 8

September carryover: 1 overdue

Do not merge this into a generic "9 October videos".

Carryover must preserve the original service period.

Historical attribution matters.

Example:

September Video #8 - delivered 3 October

8.3 Extra Content / Content Reserve

If more content is produced than required:

Example:

September requires 8
11 usable videos are produced

Then:

8 can satisfy September

Eligible surplus should automatically allocate forward to the next service period by default

Human can override forward allocation

Do not automatically publish surplus just because it exists.

Surplus content should help Aethon get ahead, not create accidental over-delivery.

8.4 Overdelivery

Aethon should detect and warn about repeated overdelivery.

Possible future insight:

Ride Inspect has received 27% more organic video output than contracted over the last 3 months.

This should feed into:

Package redesign

Pricing

Client profitability

Overservicing analysis

8.5 Client Approval Delays

When content is waiting for client approval:

Aethon's internal lateness clock should pause

Calendar reality should still continue

Publishing forecasts may still move later

The system should distinguish:

Aethon-caused delay

Client-caused delay

---

9. Contractual Allocation vs Usage

These are different concepts and must remain separate.

Example:

A content item may contractually count as:

September Content Pool #4

But its usage could be:

Instagram Organic

Facebook Organic

Meta Ad

That is still one contractual content item.

Usage must not automatically create duplicate deliverable credit.

---

10. Content Usage Types

Minimum supported usage classifications:

Organic

Paid

Organic + Paid

Reserve

Later this may expand.

Important:

Organic publishing cadence and paid-ad creative cadence are not the same.

Organic usually needs regular monthly production and posting.

Paid creatives can remain effective for multiple months and should eventually refresh based on performance/fatigue rather than automatically because a new month starts.

---

11. Delivery Health

Aethon should eventually predict whether a client is:

Ahead

On Track

Behind

At Risk

Overdue

This should not be based only on raw quantity completed today.

The long-term Delivery Health model should consider:

Contract requirement

Original service period

Current production stage

Content reserve

Historical client delivery velocity

Historical stage velocity

Current editor workload

Revision workload

Client approval waiting time

Internal target date

Current production coverage

The central prediction is:

Predicted client-approval date

That is compared to the internal target.

11.1 Historical learning

Aethon should learn over time:

Shoot -> footage prepared

Footage prepared -> first edit

Edit -> internal approval

Internal approval -> client approval

Revision duration

Client response time

Early stages:

Manual baseline

Assisted prediction

Learned prediction

11.2 Pipeline maturity matters

These are not equally risky:

Waiting for client approval

In internal review

Editing

Footage exists but editing not started

Not filmed yet

Forecasts should use the current stage and expected remaining time.

11.3 Operational KPI

A major internal KPI should be:

Shoot -> Client Approval Time

With breakdowns such as:

Footage preparation time

Editing time

Internal review time

Revision time

Client waiting time

---

12. Month-Ahead Planning

Aethon should aim to provide visibility roughly one service period ahead.

Example:

Near end of September:

October - SZ Engineers

8 required

3 already covered by reserve

5 remaining to produce

This gives:

October production coverage: 37.5%

The system should help Aethon avoid entering a month with no planned/available content.

---

13. Production Engine

The Production Engine should eventually replace much of the current Notion + manual post-shoot workflow.

Core lifecycle:

Content Idea

Ready for Shoot

Shot

Processing Footage

Editor Ready

Editing

Internal Review

Revision Required / Internal Approved

Client Review

Client Revision / Client Approved

Delivered

Scheduled

Published

Recommended content statuses:

planned

ready_for_shoot

shot

processing

editor_ready

editing

internal_review

client_review

client_approved

scheduled

published

---

14. Pre-Production

Every planned video should normally exist in Aethon before the shoot.

Content idea format:

Full script

Talking points

Rare spontaneous content can be supported later, but should not drive the primary architecture.

Pre-production requirements should include at minimum:

Content ideas ready

Location confirmed

Depending on the shoot, optionally:

Scripts ready

Talent confirmed

Gear packed

Batteries charged

Shot list ready

---

15. Footage Intake & Organisation

Long-term desired workflow:

Shoot completed

Footage dumped into Aethon intake location

Aethon detects the files

AI transcribes dialogue

AI compares transcripts against pre-existing scripts/talking points

Chronological order is used as a secondary matching signal

Clips are grouped by content item

Primary / Alternative 1 / Alternative 2 takes are selected

B-roll is identified and descriptively named

Photos are organised

Actual folders are created

Editor brief is generated

Editor tasks are created

Editor work is scheduled/prioritised

Important footage rules:

AI may move and rename physical files

AI must not delete footage without explicit human approval

Low-confidence matching must create an exception rather than silently guess

Shoots usually record one video at a time, which should be used as a useful chronological signal

B-roll is often stored separately

Example folder structure:

SZ Engineers/ 2026-09 Shoot/ Video 01 - Historical vs Active Movement/ Primary/ Alternatives/ Raw Takes/ B-Roll/ Script/ Brief/ Exports/

Aethon should preserve sensible folders even if the application disappeared.

16. B-Roll Intelligence

B-roll should be semantically named/tagged.

Example:

Original:

C0037.MP4

Possible semantic title:

Engineer inspecting exterior wall crack

The system does not need to decide every B-roll placement.

The editor retains creative control.

Long term, Aethon should make older reusable B-roll searchable across shoots.

17. Take Selection

AI may automatically choose:

Primary Take

Alternative 1

Alternative 2

These can go directly to the editor.

Saad does not need to pre-approve every take selection.

The editor can raise:

Incorrect take selection

That feedback can improve future selection logic.

18. Editing Brief Generation

Editing briefs should eventually be generated automatically using:

Script or talking points

Client-specific editing rules

Content objective

Platform

Historical approved edits

Historical revision patterns

General Aethon editing rules

Repeated revision preferences should eventually become structured rules.

Example:

If Aethon repeatedly asks:

Lower the logo

that should become a persistent client editing preference rather than being rediscovered every time.

---

19. Editor Workspace

Aethon should replace the messy editor tracker currently handled in Notion.

The editor should see a dedicated workspace showing:

Today's tasks

Priority order

Client

Content item

Deadline

Script

Brief

Primary take

Alternatives

Raw footage

B-roll

Reference assets

Client editing rules

Revision history

Initial implementation may still require manual assignments.

Long term, editor scheduling should be automatic.

---

20. Editor Scheduling & Priority

Default operational rule:

Revisions outrank untouched new edits.

Long-term scheduling should consider:

Deadline

Client revision

Deliverable risk

Publishing deadline

Dependencies

Editor workload

Human overrides

Estimated effort

Historical editing velocity

Editors should be able to flag:

This task is taking longer than expected

and the schedule should recalculate.

---

21. Editor Performance

Potential editor performance metrics:

Speed

First-pass approval rate

Number of revision rounds

Hours per video

Deadlines hit

Client feedback

Important:

Do not unfairly count all revisions against the editor.

Distinguish where possible between:

Editor-caused revision

Internal creative change

Client changing direction

Scope change

Video complexity varies, so rigid "hours per video" targets should be used carefully.

Time Doctor history may eventually help learn expected durations by content type/client.

---

22. Internal Review

Internal video review should eventually happen inside Aethon.

Desired capabilities:

Embedded playback

Timeline scrubbing

Timestamped comments

Comment replies

Resolved/unresolved state

Request revision

Approve internally

Frame.io may remain underneath technically, but Aethon should own the operating experience.

---

23. Revision History

Every revision round should remain connected to the same Content Item.

Example:

Version 1

5 internal changes requested

Version 2

4 resolved

1 new issue

Version 3

Internally approved

Client Version

2 client revisions requested

Final

Client approved

Long-term analysis should identify recurring revision causes.

---

24. Client Review

Once internally approved:

Content is exposed in the client portal.

Client can:

Approve

Request changes

Client revision comments flow back into the same revision system.

Revision work becomes high priority.

Once client approves:

Content status becomes client_approved

Deliverable is contractually delivered

Publishing Engine begins

---

25. Content Item Identity

A Content Item must remain the same record from idea to analytics.

The same object should eventually contain:

Idea

Script

Shoot

Footage

Selected takes

Brief

Editor

Versions

Reviews

Client approval

Captions

Scheduled posts

Published URLs

Analytics

Performance insights

Avoid creating disconnected records at each stage.

---

26. Publishing Engine

Once client approved:

Generate platform-specific captions

Generate YouTube title/description if required

Apply client-specific caption rules

Determine relevant platforms

Schedule posts

Publish

Track publishing success/failure

Create exception task if something fails

Possible platforms:

Instagram

Facebook

TikTok

LinkedIn

YouTube Shorts

Important:

Platform copy should be native to the platform.

Do not blindly reuse identical captions everywhere.

---

27. Publishing Cadence

Organic content should be evenly spaced based on volume.

Examples:

8 organic videos -> approximately 2/week

4 organic videos -> approximately 1/week

12 organic videos -> approximately 3/week

Paid ads are separate.

Do not assume all ads need replacement monthly.

Long-term Aethon should recommend ad refresh based on performance/fatigue.

---

28. Client-Specific Knowledge

Aethon should maintain structured client knowledge.

Examples:

Services

Target audience

Brand voice

Approved claims

Forbidden wording

CTA rules

Caption style

Editing style

Logo usage

Fonts

Brand colours

Monthly deliverables

Organic strategy

Paid strategy

Platform mix

Historical winning content

Historical revision patterns

This avoids repeatedly re-prompting AI with the same client context.

---

29. Client Portal

The client portal should be deliberately constrained.

Clients should be able to see:

Content awaiting approval

Approved content

Final deliverables

Video/view links

Monthly reports

Performance information

Relevant final assets

Clients should NOT see:

Internal delivery-risk calculations

Exact contractual counters if Aethon chooses to keep them internal

Internal deadlines

Staff workload

Overservicing

Profitability

Internal priority logic

The client portal is not the full Aethon OS.

---

30. Task Engine

Tasks should represent **human work**.

Do not create noisy tasks for automated actions such as:

Caption generated

File renamed

Report metrics pulled

Instead, tasks are for work requiring a person.

Examples:

Review video

Prepare quote

Call lead

Resolve publishing failure

Edit revision

Approve report

---

31. AI-Detected Tasks

When Aethon detects tasks from:

Emails

Meetings

Commitments

Client messages

it should normally suggest them first.

Example:

3 suggested tasks detected

Human approves before creation.

High-confidence automation can become more autonomous later.

---

32. Commitment Entity

A Commitment is not the same as a Task.

Example:

Nazif says:

We'll send that quote by Friday.

Aethon should create:

Commitment:

Client: SZ Engineers

Promise: Send Google Ads quote

Due: Friday

Source: Meeting

Then it may suggest a related Task.

The Command Centre should eventually be able to say:

3 client commitments due this week

---

33. Priority Engine

Aethon should not rely only on manual task ordering.

Long-term priority signals include:

Deadline

Deliverable risk

Client revision

Important email

Client inquiry

Meeting commitment

Publishing deadline

Blocker

Financial urgency

Human override

The system should provide human-readable reasons.

Example:

High priority because client requested revision + publication is due tomorrow + account is behind delivery pace.

No client should automatically become lower priority merely because of lower account value.

---

34. Command Centre

The Command Centre should answer:

What needs my attention?

Avoid creating a cluttered "47-widget" dashboard.

Primary information:

Critical client delivery issues

Client revision requests

Items requiring internal approval

Important commitments

Overdue tasks

Publishing failures

Serious financial issues

High-priority client inquiries

Delivery-health changes

Lower-priority work can remain visible elsewhere.

---

35. Notification Model

Notifications should remain inside Aethon OS initially.

Two conceptual levels:

High Priority

Potentially interruptive / side notification.

Examples:

Client revision request

Client inquiry

Delivery failure

Serious deadline risk

Failed publishing

Critical business issue

Lower Priority

Visible in dashboard/queue but not interruptive.

Examples:

Routine tasks

Upcoming work

Non-urgent reminders

Operational follow-up

The system should avoid becoming another noisy notification platform.

---

36. Exception System

Automation failures must never silently disappear.

If an automation fails:

Examples:

Instagram publishing failed

Drive operation failed

AI could not match footage confidently

Report data source unavailable

Then:

Create an Exception

Exceptions should:

Explain what failed

Preserve relevant context

Create human work where required

Remain auditable

Be resolvable

---

37. Audit History

Aethon requires immutable or append-only audit history for meaningful actions.

Examples:

AI scheduled this post

Saad changed priority

Deliverable definition changed

Client approved at 3:42 PM

Editor submitted revision

File was moved

Report approved

Automation failed

Audit history becomes more important as automation increases.

---

38. Communications Engine

38.1 Gmail

Aethon should eventually analyse all agency emails.

It should classify/prioritise:

Urgent

Client request

Revision

Finance

Lead

Follow-up

Commitment

FYI

The system should detect both:

Requests received

Promises/commitments Aethon makes

38.2 Meeting Intelligence

Google Meet + Granola / transcripts should eventually feed:

Meeting archive

Transcript

Summary

Decisions

Action items

Commitments

Potential upsells

Deadlines

Client requests

Example:

Omar requested LinkedIn management quote

Aethon suggests:

Prepare SZ LinkedIn proposal

---

39. CRM - Aethon's Own Sales

Aethon's own CRM is separate from client GHL accounts.

Core sales pipeline:

Lead

Contacted

Meeting Booked

Proposal

Follow-up

Won

Lost

Desired capabilities:

Stale follow-up alerts

Generated follow-up drafts

Proposal tracking

Commitment tracking

Pipeline forecasting

---

40. GoHighLevel

GoHighLevel remains primarily for client lead management.

Aethon should eventually surface common GHL data/actions such as:

Contacts

Opportunities

Pipelines

Lead source

Stage

Notes

Lead activity

Qualification

Long term:

Aethon becomes the interface; GHL becomes infrastructure.

Advanced GHL administration may still deep-link into GHL.

Do not make Aethon dependent on embedding the full GHL UI.

---

41. Meta Ads Monitoring

Meta Ads Management should be treated as an ongoing monitored service.

Aethon should eventually monitor:

Spend

CTR

CPC

Leads

Qualified leads

Cost per lead

Cost per qualified lead

Conversion patterns

Tracking anomalies

Campaign delivery

Ad fatigue

Landing-page vs lead discrepancies

Examples of future alerts:

Click volume is stable but GHL lead volume dropped sharply.

One ad is receiving high clicks but poor conversion.

Creative performance has deteriorated enough to consider refresh.

---

42. Comment Moderation

For applicable clients, Aethon may monitor social comments.

Future work may include:

Detecting unanswered comments

Prioritising relevant comments

Drafting responses

Flagging sensitive comments

Direct-message management is not a strategic future service and should not be overbuilt.

---

43. Reporting Engine

On the **1st of each month**, Aethon should generate the previous month's report.

Important:

Generate automatically. Do not automatically send.

Reports require internal approval before client delivery.

Use one Aethon master report structure with dynamic sections depending on services.

Possible data sources:

Instagram

Facebook

TikTok

LinkedIn

YouTube

Meta Organic

Meta Ads

GoHighLevel lead data

Google Ads is not currently a required reporting source.

Potential structure:

Executive Summary

Organic Content

Platform Performance

Meta Ads

Lead Quality

Key Learnings

Winning Content

Problems / Risks

Next-Month Recommendations

Reports should compare:

Current month

Previous month

3-month trend/average

Targets

Reports should include:

What happened

Why it may have happened

What should change next month

---

44. Content Intelligence Engine

Each managed client should eventually have:

44.1 Own Performance

Analyse what has historically worked.

44.2 Competitor Watchlist

Manually selected competitors/creators.

44.3 Market Discovery

Continuously discover strong-performing Australian content in relevant adjacent niches.

For SZ Engineers, research should not be limited to structural engineering.

Possible adjacent Australian niches:

Builders

Building inspectors

Trades

Construction

Property

Homeowner education

The system should adapt ideas, not copy them.

---

45. Content Performance Intelligence

Views alone are not enough.

Important metrics include:

Retention

Average watch time

Completion rate

Followers gained

Shares

Saves

Profile visits/activity

Reach

Comments

Leads

Conversions/sales where available

The system should analyse:

The actual content + analytics together.

Example future insight:

Homeowner problem hooks in the first 2 seconds produce 2.1x follower conversion vs generic educational openings.

---

46. Winning Patterns Library

Maintain per-client patterns.

Examples:

Crack-related hooks perform strongly

Direct-to-camera warnings outperform polished intros

Videos under 30 seconds retain better

Generic company-introduction videos underperform

Specific CTA structures increase profile visits

Winning patterns should feed future:

Content ideas

Scripts

Shoot planning

Editing briefs

---

47. Finance Intelligence

Xero remains the accounting system.

Aethon should surface and analyse business intelligence from Xero.

Important financial areas:

Revenue

Recurring revenue

Gross profit / contribution

Operating profit

Cash position

Cash reserve

Receivables

Client profitability

Contractor costs

Expense trends

Tax obligations

GST obligations

Tax/GST must rely on:

Xero

Accountant-confirmed information

Do not invent tax obligations independently.

---

48. Financial Targets

Preferred planning horizons:

3 months

6 months

12 months

Potential targets:

Revenue

Recurring revenue

Gross margin

Operating profit

Cash reserve

Client count

Capacity

Avoid overloading the owner dashboard with excessive ratios.

---

49. Financial Forecasting

Aethon should eventually forecast using:

Committed revenue

Current pipeline

Scenario assumptions

Scenario sandbox example:

Add 2 clients at $3,500/month

Hire another editor at $2,200/month

Add $1,000 monthly software/ad expense

Then show:

Projected revenue

Projected recurring revenue

Projected costs

Projected margin

Cash position

Capacity implications

Important:

Forecast capacity alongside money.

A business can grow revenue while destroying delivery capacity.

---

50. Time Intelligence

Time Doctor data should eventually help answer:

Which clients consume the most team capacity?

Which retainer is least profitable?

How much editing time does an average content type require?

How much time is lost to revisions?

How much of Saad's time is administrative?

At what point is another editor required?

Hiring another editor should become a concern especially when:

Missed deadlines persist

Editing backlog remains elevated

Delivery Health repeatedly deteriorates due to capacity

---

51. Team Capacity

Long-term Aethon should connect:

Client commitments

Content workload

Editor availability

Historical editing velocity

Time Doctor

Revision burden

Future insight:

At current workload, the editor team is projected to exceed safe capacity after one additional client of this profile.

---

52. Gear System

A simple internal gear module may eventually include:

Gear inventory

Purchase wishlist

Priority

Estimated cost

Reason

Expected impact

Do not build complex check-in/check-out tracking unless requirements change.

---

53. Aethon AI Operator

The AI Operator should eventually act like an executive assistant, not merely a chatbot.

Example commands:

What is falling behind this week?

What still needs approval for Ride Inspect?

Have we invoiced everything delivered this month?

Reprioritise Z's schedule because PGCC revisions are urgent.

Find an unused SZ photo and draft a LinkedIn post.

Prepare me for my meeting with Omar.

Find promises made to SZ this month that remain incomplete.

Build next month's content plan based on winning patterns.

---

54. AI Agent Architecture

One user-facing Aethon AI may route to specialised underlying agents.

Possible agents:

Operations Agent

Production Agent

Content Strategy Agent

Finance Agent

CRM / Sales Agent

Reporting Agent

Research Agent

The routing can be invisible to the user.

---

55. Automation Risk Levels

Automation should be divided by risk.

Fully Automatic

Examples:

Generate captions

Organise files

Create editor tasks

Transcribe footage

Calculate deliverable progress

Pull analytics

One-Click Approval

Examples:

Send client email

Create invoice draft

Schedule unusual content

Accept AI-detected task/commitment

Explicit Human Control

Examples:

Payments

Deleting important data

Financial commitments

Major scope changes

Deleting footage

---

56. Aethon Health

Aethon Health should be a management layer, not one gimmicky 0-100 score.

Health categories:

Delivery Health

Are client commitments being fulfilled?

Client Health

Outstanding requests, approvals, complaints, revision load, communication issues.

Production Health

Editing backlog, review bottlenecks, footage waiting, revision load.

Capacity Health

Team workload and deadline pressure.

Financial Health

Revenue, cash, margin, expenses, receivables.

Sales Health

Pipeline, stale opportunities, follow-ups, forecast.

Marketing Health

Client campaign/content anomalies.

Philosophy:

Do not show everything. Show what changed, what is wrong, and what needs a decision.

---

57. Core Data Model

The following entities are expected to exist eventually.

Not all need to be implemented immediately.

Core:

Client

Contact

Engagement

Contract

Service Period

Deliverable Definition

Deliverable Instance

Content Item

Shoot

Asset

Brief

Task

Assignment

Commitment

Review

Revision

Approval

Publication

Platform

Service Monitor

Metric Snapshot

Report

Alert

Exception

Meeting

Communication

Lead

Opportunity

Campaign

Invoice

Expense

Time Entry

Forecast Scenario

Agent Action

Audit Event

---

58. Foundational Relationships

Conceptually:

Client -> Engagement -> Contract -> Service Period -> Deliverable Instance -> Content Item

Client -> Shoot -> Content Item -> Assets

Content Item -> Brief -> Assignment -> Review -> Revision -> Approval -> Publication -> Metrics

Meeting / Email -> Commitment -> Suggested Task

Engagement -> Finance -> Time -> Profitability -> Health

---

59. Event-Driven Architecture

Aethon should eventually use meaningful domain events rather than disconnected ad-hoc automations.

Examples:

contract.uploaded

deliverables.confirmed

service_period.created

shoot.created

shoot.completed

footage.uploaded

footage.processed

brief.generated

edit.submitted

internal_review.requested

internal_revision.requested

internal_approval.completed

client_review.requested

client_revision.requested

client_approval.completed

deliverable.completed

caption.generated

content.scheduled

content.published

publication.failed

meeting.completed

email.action_detected

commitment.created

deliverable.at_risk

invoice.overdue

automation.failed

Other systems react to these events.

Avoid "random Zap"-style coupling inside the core architecture.

---

60. External Integration Strategy

Use three integration levels.

Native

Build directly into Aethon where it is core internal workflow.

Examples:

Clients

Deliverables

Tasks

Content lifecycle

Editor workspace

Command Centre

Health

Audit

API-Driven

Use official APIs where practical.

Examples:

Google

Xero

Meta

GoHighLevel

YouTube

Social platforms

Agent-Driven

Use computer-use / Codex-style agents when an application does not expose required functionality cleanly.

Agent-driven workflows are a fallback or last-mile option, not the default architecture.

---

61. Current Platforms and Intended Role

Notion

Likely replaced substantially for:

Tasks

Agency hub

Editor portal

Deliverables

Client operating records

Google Drive

Remain storage infrastructure.

Aethon should make Drive feel native.

Gmail

Remain mail provider.

Aethon adds prioritisation/intelligence.

Google Calendar

Remain scheduling infrastructure.

Aethon surfaces and uses calendar data.

Google Meet

Remain meeting platform.

Granola

Meeting transcription/summary source.

Frame.io

May remain review/video infrastructure underneath.

Aethon should increasingly own the workflow.

Canva

Remain design tool.

GoHighLevel

Remain client CRM infrastructure.

Xero

Remain accounting system of record.

Meta Ads Manager / Business Suite

Remain paid/social management infrastructure.

Instagram / Facebook / TikTok / LinkedIn / YouTube

Remain publishing/distribution platforms.

Time Doctor

Remain time-tracking source.

CapCut

Remain editing tool where needed.

AI tools

Used for automation, reasoning, coding, content processing, and agent workflows.

---

62. UX Principles

Aethon OS should feel:

Fast

Clean

Professional

Restrained

Information-dense without clutter

Operationally focused

Avoid:

Generic admin-template look

Excessive gradients

Glassmorphism

Neon AI aesthetic

Oversized decorative cards

Unnecessary animations

Dashboard clutter

Aethon brand direction:

Dark navy

White

Neutral greys

Helvetica-like clean typography where appropriate

---

63. Primary Application Areas

Long-term navigation may include:

Command Centre

Clients

Deliverables

Production

Content

Editor

Calendar

CRM

Inbox

Meetings

Publishing

Reports

Finance

Team

Gear

AI Operator

Settings

Client-specific pages may include:

Overview

Engagements

Deliverables

Content

Shoots

Files

Tasks

Reports

Leads

Ads

Finance

Do not expose all sections before they exist meaningfully.

---

64. Recommended Incremental Build Order

This is a roadmap, not a rigid contract.

Milestone 0 - Development Environment

Windows/Tauri toolchain

Git

Node

Rust

Visual C++ build tools

WebView2

Project scaffold

Local repo

Milestone 1 - Core Operational Foundation

Build:

Client

Engagement

Service Period

Deliverable Definition

Deliverable Instance

Content Item

Audit Event

SQLite persistence

Clients screen

Deliverables screen

Content list

Content pipeline

Basic Command Centre

First vertical slice:

Client -> Engagement -> Monthly Deliverable -> Content Item -> Production Status -> Client Approved -> Deliverable Progress

Milestone 2 - Editor & Production Workflow

Build:

Shoot entity

Briefs

Editor workspace

Assignments

Internal review flow

Revision history

Better task system

Priority foundation

Initially manual where necessary.

Milestone 3 - Footage Automation

Build:

Local intake watcher

Transcription

Script/talking-point matching

Chronological matching

Primary / Alt takes

B-roll naming

Folder creation

Brief generation

Exception handling

Milestone 4 - Client Portal

Build:

Client review

Approve / request changes

Client revision flow

Limited client-facing content/report access

Milestone 5 - Publishing Engine

Build:

Caption generation

Platform variants

Organic cadence

Scheduling

Publishing

Failure/exception handling

Milestone 6 - Google / Communications

Build:

Gmail

Calendar

Meet/Granola

Email prioritisation

Meeting archive

Suggested tasks

Commitments

Milestone 7 - Reporting

Build:

Platform metrics

Meta Ads

GHL

Monthly report generation

Month-over-month comparison

3-month trends

Recommendations

Approval workflow

Milestone 8 - Finance & Business Intelligence

Build:

Xero

Revenue

Profitability

Expenses

Receivables

Tax/GST display

Forecasting

Capacity linkage

Business Health

Milestone 9 - Content Intelligence

Build:

Own performance analysis

Australian-market research

Competitor watchlists

Winning patterns

Content recommendations

Milestone 10 - AI Operator

Build:

Conversational control

Agent routing

Executive-assistant behaviours

Safe action approvals

Milestone 11 - Multi-User

Only after the single-user workflow is mature:

Shared backend

Authentication

Roles

Permissions

Team accounts

Multi-device synchronisation

Installers / updates

---

65. Current Implementation Status

Important: Codex must audit the actual repository before trusting this section.

Current known state from planning conversation:

Product architecture: substantially defined

Initial development environment: intended to be set up on Saad's Windows PC

Initial stack: Tauri 2 + React + TypeScript + Vite + SQLite

Multi-user: intentionally deferred

Cloud backend: intentionally deferred

Repository implementation status:

☐ Development environment verified

☐ Tauri application scaffolded

☐ SQLite configured

☐ Migrations configured

☐ Client entity implemented

☐ Engagement entity implemented

☐ Service Period implemented

☐ Deliverable Definition implemented

☐ Deliverable Instance implemented

☐ Content Item implemented

☐ Audit Event implemented

☐ Clients UI implemented

☐ Deliverables UI implemented

☐ Content list implemented

☐ Content pipeline implemented

☐ Client approval updates deliverable progress

☐ Data persists across restart

☐ Seed/demo data implemented

☐ Command Centre foundation implemented

☐ Editor workspace implemented

☐ Shoot workflow implemented

☐ Review/revision system implemented

☐ Footage processing implemented

☐ Client portal implemented

☐ Publishing engine implemented

☐ Communications integrations implemented

☐ Reporting engine implemented

☐ Finance intelligence implemented

☐ Content intelligence implemented

☐ AI Operator implemented

☐ Multi-user architecture implemented

Codex must update this checklist based on the actual repo.

66. Definition of "Implemented"

A feature should not be marked implemented merely because:

A page exists

A component compiles

Placeholder data appears

To mark implemented, Codex should verify:

Real local persistence where required

Core workflow functions end-to-end

Important validation exists

Errors are handled reasonably

App still launches

Existing relevant behaviour still works

Feature is usable in real Aethon operations

Use "Partially implemented" when appropriate.

67. Build History

Codex should append meaningful milestones here.

Format:

YYYY-MM-DD - Milestone / Feature

Implemented:

...

Changed:

...

Known limitations:

...

Next recommended step:

...

68. Known Locked Decisions

These should not be changed casually.

Aethon OS begins as a local Windows application.

SQLite is the initial source of truth.

Deliverables are the central operational spine.

Client approval counts as contractual delivery.

Scheduling/publishing continue after delivery.

Carryover keeps original service-period attribution.

Carryover is displayed separately from current-period commitments.

Surplus content should allocate forward by default.

Overdelivery should be detectable.

As-needed ceilings are maximums, not required targets.

Client approval delay pauses Aethon's internal delivery clock.

One content item should not automatically count against two contractual obligations.

Every planned video should normally exist before the shoot.

Footage matching should use transcript-semantic matching + chronology.

AI may move/rename footage but may not delete it without human approval.

AI-selected takes may go directly to the editor.

Editing briefs should be generated automatically later.

Internal review should eventually happen inside Aethon.

Client revisions must remain in structured history.

Revisions generally outrank new edits.

Tasks are for human work, not every automation.

AI-detected tasks/commitments initially require approval.

All clients are important; client value alone should not control priority.

Monthly reports are generated automatically but require internal approval before sending.

Client portal does not expose sensitive internal delivery/profitability information.

Xero remains accounting source of truth.

Tax/GST information must rely on Xero/accountant-confirmed information.

Finance planning horizons are 3, 6, and 12 months.

Aethon AI should behave like an executive assistant.

Specialised agents may exist under one interface.

Notifications stay in Aethon initially.

Automation failures create exceptions.

Audit history is required.

Learning/resources module is not currently in scope.

Complex gear check-in/out is not currently in scope.

Multi-user/cloud architecture should be deferred until the single-user operating model is proven.

69. Things We Deliberately Do Not Want Codex To Do Yet

Unless a milestone explicitly calls for them:

Do not build a cloud backend prematurely.

Do not add authentication just because "production apps need auth".

Do not convert SQLite to Supabase/Firebase/Postgres without a deliberate migration decision.

Do not create a generic SaaS architecture.

Do not optimise for external customers.

Do not create dozens of abstract services before they are needed.

Do not build integrations before the core workflow they support exists.

Do not implement AI everywhere just because it is possible.

Do not replace specialist software without a clear operational reason.

Do not create notification spam.

Do not silently automate high-risk actions.

Do not delete footage automatically.

Do not assume every client has the same package structure.

Do not hard-code SZ Engineers or Ride Inspect logic as universal business rules.

---

70. Codex End-of-Task Protocol

After every meaningful implementation task, Codex should:

Run relevant checks/tests.

Launch the app when practical.

Confirm existing workflows still function.

Summarise:

What was changed

What remains incomplete

Any architecture decisions made

Any deviations from this document

Update:

Implementation Status

Build History

Recommend the **single next logical milestone**, not ten unrelated ideas.

Do not automatically start the next major milestone without instruction.

---

71. Product Success Criteria

Aethon OS succeeds if it progressively enables Aethon Studios to:

Stop forgetting client deliverables

See delivery risk early

Avoid accidental overdelivery

Work ahead of monthly obligations

Reduce post-shoot admin

Reduce editor coordination overhead

Make revisions faster

Centralise client approvals

Automate captions and publishing

Consolidate reporting

Detect important client emails/commitments

Surface business priorities automatically

Understand profitability and capacity

Know when hiring is required

Learn what content performs

Forecast growth scenarios

Use AI to operate the business, not just answer questions

The long-term target is:

Aethon leadership should spend minimal time manually coordinating routine agency administration.

---

72. Final Architecture Philosophy

Aethon OS should not become another bloated project-management application.

It should become:

An operations control system for Aethon Studios.

The system should continuously understand:

What Aethon promised

What has happened

What should happen next

What is at risk

What requires human judgement

What can be automated safely

The build strategy is:

Digitise the correct workflow -> use it -> observe friction -> improve it -> automate it -> repeat.

Do not wait for the perfect architecture before building.

Do not build blindly without architecture.

Maintain the balance through this document.